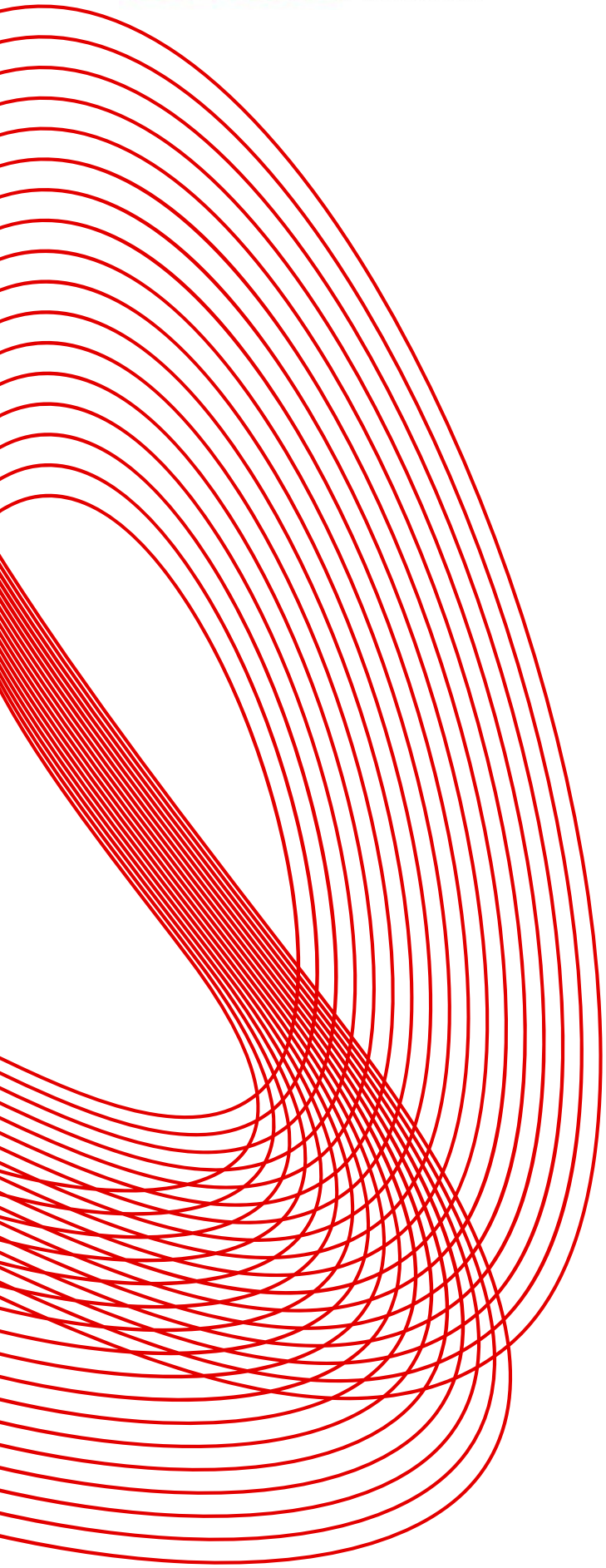
INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES STRASBOURG

S9



S-IoT Project Report :

Drone swarm

Written by :

Emilien PEYON

Noé RAMDANI

Class of 2022-2027

Contents

I Main **4**
1 UWB . . . 4
A Algorithm . . . 4
B Calibration . . . 4
C Test . . . 4
D Filter . . . 6
2 Communication . . . 8
A Type of communication . . . 8
B Choice . . . 8
C ESP-NOW implementation . . . 9
3 Data processing . . . 10
4 Visualization . . . 11
A Matlab . . . 11
B Architecture . . . 11
5 Simulation . . . 12
A ArduPilot and Mission Planer . . . 12
B Architecture . . . 12
C Result . . . 13
D Simulation possibilities . . . 13
6 Outdoor Test . . . 14

List of Figures

I.1 Schéma de la configuration face à face horizontale 4
I.2 Schema of the paralel configuration 5
I.3 Schema of the paralel configuration 5
I.4 Schema of the paralel configuration 5
I.5 Code for NLOS Filter 6
I.6 Code for NLOS Filter 6
I.7 Code for NLOS Filter 7
I.8 Code for send messsage using ESP-NOW 8
I.9 Drone communication package 9
I.10 Drone communication package 9
I.11 Visualization architecture 11
I.12 Simulation architecture 13

I - Main

1 - UWB

A - Algorithm

Say that we take DS-TWR et TDMA

B - Calibration

Tableau avec les références

C - Test

Face to Face horizontaly

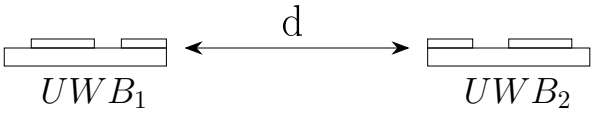


Figure I.1: Schéma de la configuration face à face horizontale

Réelle (cm)	Mesurée (cm)	Erreur (cm)
69,4	61	-8,4
70	62,2	-7,8
75	69,6	-5,4
80	73,9	-6,1
82,6	76,1	-6,5
85	79,9	-5,1
90	86,6	-3,4
91	87	-4
95	92,2	-2,8
100	96,5	-3,5
100	96,8	-3,2
105	102,6	-2,4
107,3	106,4	-0,9
110	108,9	-1,1
115	113,4	-1,6
120,5	120,9	0,4
125	126,4	1,4
130	129,6	-0,4
132	133,7	1,7

Réelle (cm)	Mesurée (cm)	Erreur (cm)
135	133,6	-1,4
140	138,6	-1,4
145	144,9	-0,1
147,4	150	2,6
150	151,7	1,7
155	160,3	5,3
155,5	161	5,5
160	165,7	5,7
164,7	171,8	7,1
165	172,2	7,2
167	172,8	5,8
170	177,2	7,2
175	180,3	5,3
176	183,8	7,8
180	184,3	4,3
185	188,3	3,3
189,6	197,9	8,3
190	191,9	1,9

Table I.1: Mesure face à face de façon horizontale

Paralel module

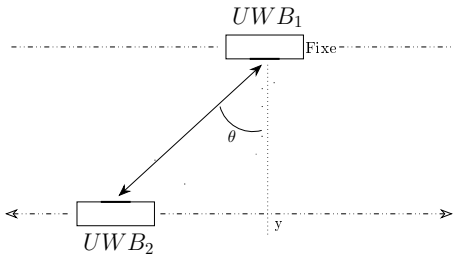


Figure I.2: Schema of the paralel configuration

Décalage (cm)	Angle (°)	Dist. réelle (cm)	Dist. mesurée (cm)	Erreur
0	0	120	117	3
10	4,78	121	118,4	2,6
20	9,59	121,5	118,9	2,6
30	14,48	122,5	119,1	3,4
40	19,47	125,8	124,8	1
50	24,62	128,5	128,7	-0,2
60	30,00	132,5	134,4	-1,9
70	35,69	137	142,6	-5,6

Table I.2: Measure of the paralel configuration

Rayon configuration

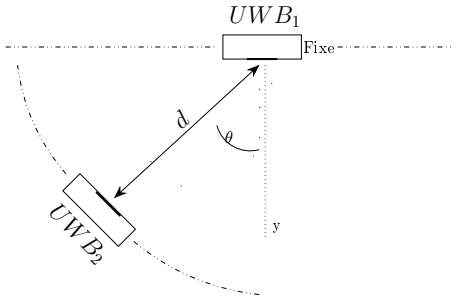


Figure I.3: Schema of the paralel configuration

Angle(°)	Distance réelle (cm)	Distance mesurée (cm)
10	120	116,7
20	120	118,4
30	120	119,5
40	120	122,8
50	120	125,9
60	120	124,2
70	120	123
80	120	124
90	120	130

Table I.3: Measure of the rayon configuration

Face to Back

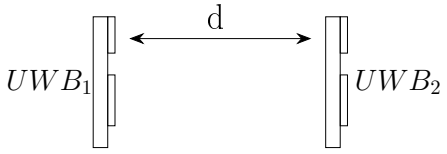


Figure I.4: Schema of the paralel configuration

Distance réelle (cm)	Distance mesurée (cm)	Erreur
70	97	-27
80	109,8	-29,8
90	122,8	-32,8
100	133,3	-33,3
110	143,5	-33,5
120	152	-32
130	161,6	-31,6
140	177,2	-37,2
150	188	-38
160	192,2	-32,2
170	208,8	-38,8

Table I.4: Measure of the rayon configuration

Test de positionement, ajouter les tableaux et les schéma (cote à cote)

D - Filter

Polynomial filter

```c
void polynomial_filter(float* d) {
    float raw_cm = *d * 100.0f;

    // Equation d'erreur = a5*x^4 + a4x^3 + a3*x^2 + a2*x + a1
    const float a5 = -1.004e-6f;
    const float a4 = 0.0005059f;
    const float a3 = -0.09263f;
    const float a2 = 7.398f;
    const float a1 = -221.7;

    float estimated_error_cm = a1 + raw_cm * (a2 + raw_cm * (a3 + raw_cm * (a4 + raw_cm * a5)));

    float corrected_cm = raw_cm - estimated_error_cm;

    if (corrected_cm < 0.0f) {
        corrected_cm = 0.0f;
    }

    *d = corrected_cm / 100.0f;
}
```

Figure I.5: Code for NLOS Filter

NLOS Filter

```c
typedef struct {
    float firstPathPower;
    float receivePower;
    float firstPathAmp1;
} UWB_Diagnostics;

bool NLOS_filter(UWB_Diagnostics diag) {

    float min_fpp_threshold = -85.0f;
    if (diag.firstPathPower < min_fpp_threshold) {
        return false;
    }

    float power_diff = fabsf(diag.receivePower - diag.firstPathPower);
    float max_power_diff = 6.0f;

    if (power_diff > max_power_diff) {
        return false;
    }

    return true;
}
```

Figure I.6: Code for NLOS Filter

Innovation Gate Filter

```
bool innovationGate_filter(float current_d, float prev_d, float max_speed, float dt) {

    float max_possible_delta = max_speed * dt;

    float sensor_noise_margin = 0.2f;
    float allowed_change = max_possible_delta + sensor_noise_margin;

    float actual_change = fabsf(current_d - prev_d);

    if (actual_change > allowed_change) {
        return false;
    }

    return true;
}
```

Figure I.7: Code for NLOS Filter

2 - Communication

In the project, we have to differents use for communication, first communication is used to made each drone talk with each other to maintain the shape but also adapt themselves to the other. Second communication is the data's descent from the drone to the base to permit visualition of the swarm and command drone.

A - Type of communication

We have studied and made research of 3 different communication protocol.

Xbee

The first communication protocol, we make research about is Xbee because it was recommended by the subject. It works well for large range purpose (multiple km range). It has been already implemented by others groups, and by the research team. So, the pipeline was already desin. Otherwise, it needed a supplementary module. Which complexifie the communication management. And, it is in the contrary of the lighweight solution we want to create.

UWB

Second communication protocol was to use the UWB as a sensor and also as communication module. It has been researched and implemented in real life. But during the first test, we have seen some capability reduction due to extension of the message which create a reduction of the frequency. Otherwise, it still a good idea cause it only use the UWB module, that reduce the complexity of the system.

ESP-NOW

The third communication protocol is ESP-NOW. A specific communication protocol made by Espressif (ESP32 manufacturer) for ESP32. It uses the Wi-Fi antenna and it bandwith of 2.4Ghz. Optimize for ESP because code his handled by the manufacturer, and it pass before every other task on the Wi-Fi instruction pile. It is range can reach more than 150m.

B - Choice

We choose ESP-NOW has the protocol for two main raison : it doesn't need new hardware implementation. And, it responds to both communication purpose of the project. Then, has said during the presentation, the code has been optimized and write by the manufacturer which facilitate it implementation. For exemple, send data is only this command :

```
1  esp_now_send(MAC_ADRESS, data, lenght_of_data);
```

Figure I.8: Code for send messsage using ESP-NOW

Has said before, range reach 150 meters allows us to create a base. It only ask us, to have a ESP32, connect to our computer.

C - ESP-NOW implementation

To implement ESP-NOW, we had to think about the data we want to exchange. For our first test, we are using two different packages for two different purpose.
First for inter-drone communication :

```
typedef struct __attribute__((packed)) {
  uint8_t  id;
  uint32_t t_ms;
  float d;
  float z;
  GPS gps;
  Attitude a;
} Pkt;
```

Figure I.9: Drone communication package

And for the base to drone communcation we have this package :

```
typedef struct __attribute__((packed)) {
  uint8_t  id;
  uint32_t t_ms;
  uint8_t commande;
  float variable;
} Pkt_base;
```

Figure I.10: Drone communication package

3 - Data processing

4 - Visualization

Visualization permit to obtain precise measurement and result from test in major case drone will fly outdoor at 10 meters away from us. So, vizualise permit to precisely see the attitude of the drone, the real shape positioning. It can permit to detect error, or problematic point.
To visualise, we choose the sofware Matlab. And had the objectif to be able to see the attitude, the shape position and also look at the data coming from the drone.

A - Matlab

Matlab is a general tool developped by the MathWorks. It has many functions such as mathematics calculator, simulator, and also a corrector designer. Based on add-on and toolbox, it can be usefull for many differents purpose. It also permits to generate an interface.

B - Architecture

The visualization follow the next architecture :

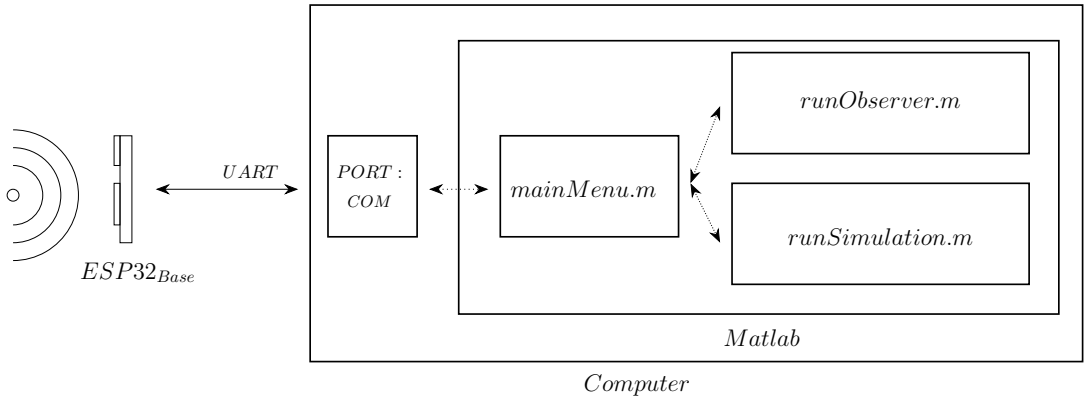


Figure I.11: Visualization architecture

Let's explain each different bloc and its role :

- ESP32 : Relay between ESP-NOW protocol and the UART Port of the computer, which permit to make it visible for the matlab code.
- COM PORT: Physical bridge between the computer and the ESP32
- mainMenu : General window which redirect for the next window
- runObserver : Window to observe UART communication on the PORT (Debug utility)
- runSimulation : Main Window where we visualise attitude of the drone, shape placement and data reader

5 - Simulation

On this project, outdoor tests are very rare due to logistical problem. However, we need to verify our work such as the corrector. To do so, we developed a simulation protocol using the real MAVLink messages, and same data as our drone.
In this part, we will first present the main simulation tools, ArduPilot and Mission planer, second, explain the implementation, then present the result we obtained and finally the simulation possibilities.

A - ArduPilot and Mission Planer

To do drone simulation, we have to select the same architecture as our drone on the pixhwak 4. In the drone industry, two differents architectures are existing: Ardupilot and PX4. In our case, pixhwak 4 is using the ArduPilot architectures.
Then, after selecting the architecture, we have to use a software to visualise and simulate drone. We choose mission planer because we have already work with it on different project.
Let's present both tools:

ArduPilot

ArduPilot was created in 2007 by the DIY Drones community. Originally developed for Arduino boards, it has since become a general-purpose, open-source autopilot. It can control many types of vehicles (copters, planes, rovers, submarines, etc.). ArduPilot provides autopilot functions, receives data from sensors, and fuses it using an Extended Kalman Filter (EKF) [ref]. It also allows users to create pre-planned missions. To operate, it uses the MAVLink protocol to communicate with the drone and facilitate control. In our case, an ESP32 uses the MAVLink library to send MAVLink messages that command the drone's movements.

Mission planer

Mission planer is a representative tool for copters such as drone. Created by Michael Oborne to help users of ArduPilot architecture to vizualise their drone and command them. The software also permits to declare multiple drone so create a swarm. For our project, it will be used as a simulation field.

B - Architecture

The simulation works with a very specific architecture cause tools like ArduPilot works under Linux system. The architecture is exposed in the figure below.

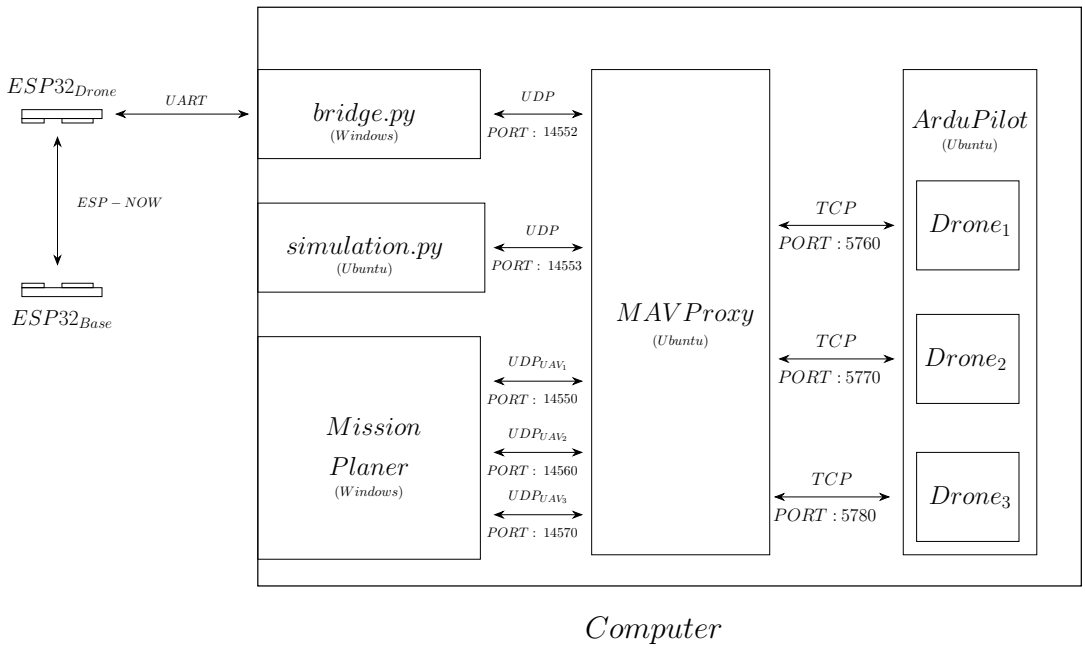


Figure I.12: Simulation architecture

Let's explain each different bloc and its role :

- $ESP - 32_{Base}$: Represent order send from the base for exemple takeoff
- $ESP - 32_{Drone}$: Represent the main drone where code we want to test is running
- ArduPilot : Represent the pixhawk of our drone. Here, we are simulating 3 of them.
- Bridge.py : Code made to redirect data from the UART connection to UDP connection
- MAVProxy : Manage all entries and exit permit to link the drone entries redirect them to the Ardupilot and also permit to redirect exit from ArduPilot (GPS, speed, etc) to Mission Planer.
- Mission Planer : Use to visualise our swarm drone based on data from ArduPilot
- Simulation.py : Python scrip to simulate $Drone_2$ and $Drone_3$

C - Result

Add after fist Test

D - Simulation possibilities

Add after fist Test

6 - Outdoor Test